Bluetooth LE Advertisement Example Application Note

Revision History

Date	Version	Description
November 23, 2020	0.50	Initial version created.
December 2, 2020	0.51	Corrected typos.
April 14, 2021	0.52	Updated format, no content change.
September 29, 2021	0.53	Added 3.3 Dynamic Advertisement Scenario

CONFIDENTIAL

©2020-2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc.
All other trademarks are the properties of their respective holders. This document is subject to change without notice.

Table of Contents

1 Overview 3

2 Quick Start 3
- 2.1 Configuration 3
  - 2.1.1 Flash NVDS Tags 3
  - 2.1.2 Configuration Header File 4
- 2.2 Sanity Check Of Advertising Configuration 5
- 2.3 General Registration 5
  - 2.3.1 GAP Callbacks 5
  - 2.3.2 Advertisement Callback 5
- 2.4 Advertising State Transition 7

3 BLE_adv Example 8
- 3.1 Introduction 8
- 3.2 Application State Machine 8
- 3.3 Dynamic Advertisement Scenario 9
  - 3.3.1 Configuration 9
  - 3.3.2 Update Advertisement Flow 9

CONFIDENTIAL

List of Figures

Figure 1 - Configuration Header File
Figure 2 - Advertising State Transition Diagram
Figure 3 - Application State Machine Flow Diagram

List of Tables

Table 1 - API and Corresponding NVDS Tags
Table 2 - Complete Local Name Update Rule

1 Overview

BLE_adv is a flash based application demonstrating Bluetooth Low Energy advertiser capability. This application note offers a set of preconfigured beacon examples as well as allows customization of beacon configuration parameters and payload.

2 Quick Start

To build and run BLE_adv application using default example:

```
$ cd <SDK_ROOT>/atmosic_sdk/platform/atm2/<platform>/examples/BLE_adv
$ make run_all
```

Default beacon configuration sends Eddystone beacon (non connectable and scannable) every 100 ms with “Atmosic Beacon” as local name.

2.1 Configuration

Configuration of the BLE_adv example is provided in the following sections.

2.1.1 Flash NVDS Tags

The Non Volatile Data Storage (NVDS) settings in flash allow for rapid development and testing of OTP_beacon configuration parameters without having to burn through the OTP in the devices. See ../OTP_beacon/README for more information.

Some predefined beacon profiles have also been defined. Refer to the reference_beacon_XXX in examples/OTP_beacon/reference_beacons.mk file. Use following command to build and run any of these examples

```
$ make REF_BCN:=XXX run_all (example: XXX=simple_beacon)
```

Note that the BD address will be picked up from the NVDS as well if included in the definition of the XXX target. The predefined beacons can be used as a starting point for more advanced beacon applications such as those that are connectable and those that gather information from sensors and embed the data in advertisements.

Below are the NVDS tags related to the Bluetooth LE advertisement. Refer to the Atmosic SDK Reference Manual (available on the Atmosic support site) for more information about the Flash NVDS tags setting.

- 05-APP_BLE_ACT_STRT_CMD
- 06-APP_BLE_ACT_CRT_CMD
- 09-APP_BLE_RSTRT_DUR
- 0b-APP_BLE_ADV_DATA
- 0c-APP_BLE_SCAN_RESP_DATA

CONFIDENTIAL

The Flash NVDS tags can be modified through the Atmosic configurator (refer to the tools/README.configurator) or through the BLE_adv example makefile using the 'make' command. Table 1 below shows the APIs available to query the corresponding NVDS tags.

API	Tag Data
atm_adv_create_param_nvds	06-APP_BLE_ACT_CRT_CMD
atm_adv_start_param_nvds	05-APP_BLE_ACT_STRT_CMD
atm_adv_advdata_param_nvds	0b-APP_BLE_ADV_DATA
atm_adv_scandata_param_nvds	0c-APP_BLE_SCAN_RESP_DATA
nvds_get(NV_TAG_APP_BLE_RSTRT_DUR..)	09-APP_BLE_RSTRT_DUR

Table 1 - API and Corresponding NVDS Tags

2.1.2 Configuration Header File

The Atmosic framework provides a predefined set of Bluetooth LE advertisement parameters through the "config_adv_params.h" header file in the BLE_adv example. To include the configuration header file for the predefined parameters, modify the CFLAGS as follows:

CFLAGS +=DGAP_ADV_PARM_NAME="config_adv_params.h"

The default configuration header file is compatible with Keil configuration wizard. Figure 1 shows the snapshot of the file when opened using the Keil IDE.

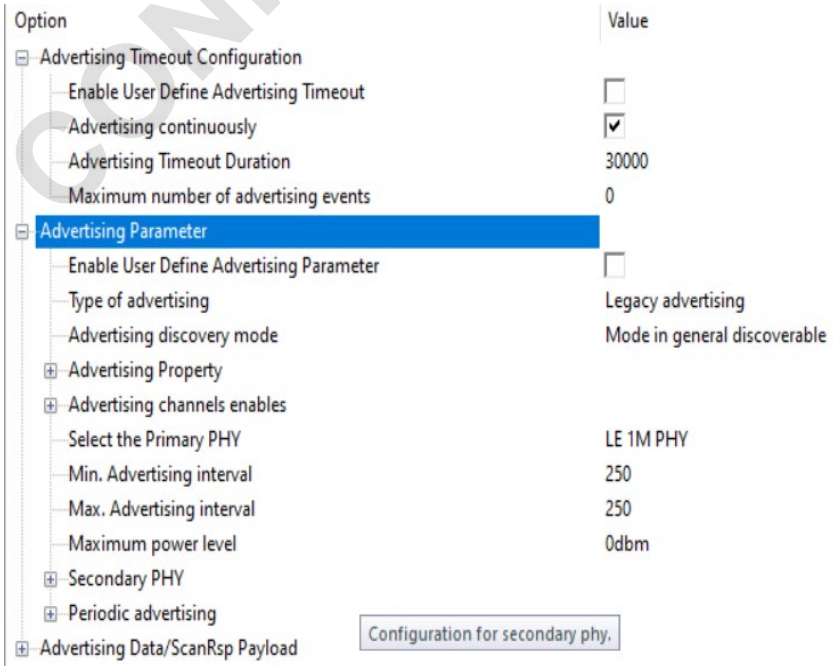


Figure 1 - Configuration Header File

CONFIDENTIAL

The header file includes three major sections as below. The sub items within these major sections will be enabled after turning on the corresponding major section.

- Advertising Timeout Configuration (*ENABLE_USER_ADV_TIMEOUT*)
- Advertising Parameter (*ENABLE_USER_ADV_PARAM_SETTING*)
- Advertising Data/ScanRsp Payload (*ENABLE_USER_ADV_DATA_SCANRSP*)

The settings of the predefined header file will be used in default_adv_create_param, default_adv_start_param and default_set_adv_data . Those three variables are in the atm_adv_param.c source file.

NOTE: If you want to create two advertisements, you need to overwrite the CFG_GAP_ADV_MAX_INST definition. The two advertisements cannot run concurrently.

Refer to the configuration header file for details on setting the above parameters.

2.2 Sanity Check Of Advertising Configuration

The Atmosic framework provides Bluetooth LE advertising parameters sanity check feature to make sure the parameters are correct. If the parameters are incorrect, it will print the error message in the debug console. Refer to atm_adv_create_sanity() and atm_adv_set_data_sanity() of atm_adv.c.

2.3 General Registration

There are two components which GAP and Advertising need to implement when enabling the Bluetooth LE advertisement feature. The following section describes the minimal requirements for each component.

2.3.1 GAP Callbacks

The Bluetooth LE advertisement example uses the GAP initial confirm callback function as the APP starts. This means that the first thing to implement is the initial confirm callback function of the type atm_gap_cbs_t, and register it to the system through atm_gap_start() function. Refer to the implementation of ble_adv_init() and definition of gap_callbacks for additional information.

2.3.2 Advertisement Callback

An advertisement callback function can be registered through the atm_adv_reg() API. This enables one to be notified of the advertising state machine transitions.

CONFIDENTIAL

```
static void adv_state_change(atm_state_e state, uint8_t act_idx, enum hl_err status)
{
    switch (state) {
...
        case ATM_ADV_CREATED: {
            ble_adv_create_cfm(act_idx, status);
        } break;
....
}
atm_adv_reg(adv_state_change);
```

Following list is all adv states in the atm_adv_state_t definition.

```
/// ADV state.
typedef enum {
    /// Advertising state machine is in idle state.
    ATM_ADV_IDLE,
    /// Advertising state machine is in creating state.
    ATM_ADV_CREATING,
    /// Successfully created, data not set.
    ATM_ADV_CREATED,
    /// ADV data is under setting.
    ATM_ADV_ADVDATA_SETTING,
    /// ADV data is set.
    ATM_ADV_ADVDATA_DONE,
    /// Scan data is under setting.
    ATM_ADV_SCANDATA_SETTING,
    /// Scan data is set.
    ATM_ADV_SCANDATA_DONE,
    /// Advertising state machine is in off state.
    ATM_ADV_OFF,
    /// Advertising state machine is in starting state.
    ATM_ADV_STARTING,
    /// Advertising state machine is in on state.
    ATM_ADV_ON,
    /// Advertising state machine is in stoping state.
    ATM_ADV_STOPPING,
    /// Advertising state machine is in deleting state.
    ATM_ADV_DELETING,
    /// Delete created advertising successfully.
    ATM_ADV_DELETED
} atm_adv_state_t;
```

2.4 Advertising State Transition

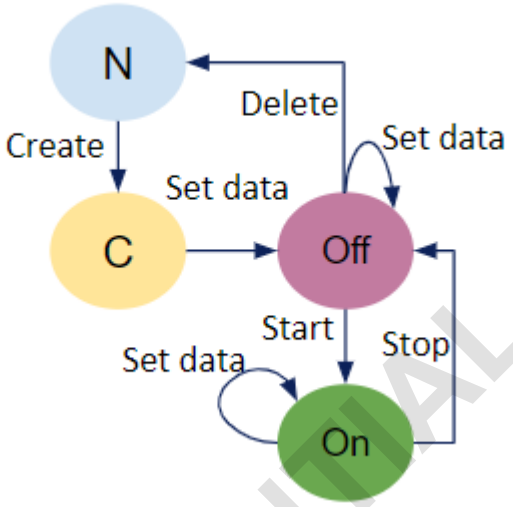


Figure 2 - Advertising State Transition Diagram

Refer to the atm_adv.h header file for the APIx exposed to the BLE_adv app.

CONFIDENTIAL

3 BLE_adv Example

3.1 Introduction

This example initiates the Bluetooth LE advertising, and applies the parameter and payload settings loaded from Flash NVDS tag, header file or use default settings based on the compile flags. The API, atm_adv_create_param_nvds() can be used to get the NVDS settings in flash. The API atm_adv_create_param_get() can be used to get the settings from the header file or use defaults in the code. There are three advertising settings - ***Advertising Parameter***, ***Advertising Data/ ScanRsp Payload*** and ***Advertising Timeout Configuration*** that are used for advertising configuration.

The application code can use these compile definitions (refer to the configuration header file included with the example - 2.1.2 Configuration Header File) to make a decision about using the settings either from Flash NVDS tag or the predefined header file. Refer to the ble_adv_init() function for additional details on how to establish different parameters.

3.2 Application State Machine

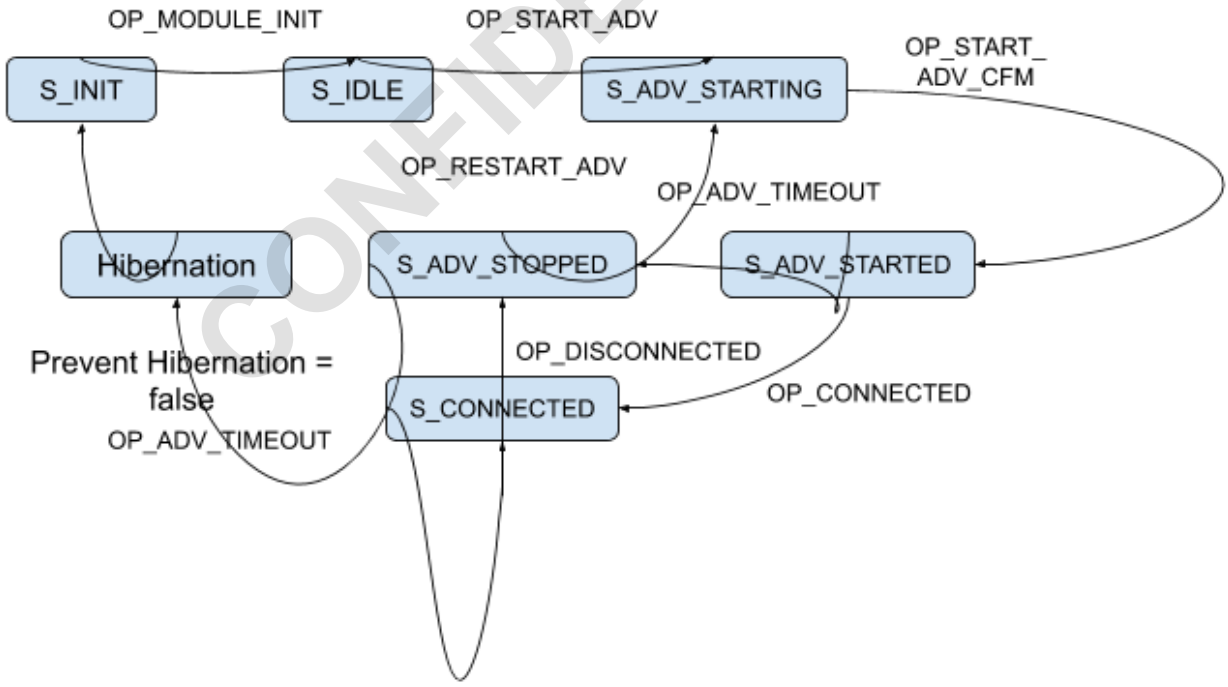


Figure 3 - Application State Machine Flow Diagram

Refer to the definition of s_tbl for more information on the states and state machine transitions.

CONFIDENTIAL

3.3 Dynamic Advertisement Scenario

The BLE_adv example also can configure dynamic change ***Advertising Data/ ScanRsp Payload.*** Following sections introduce how to configure and how it works.

3.3.1 Configuration

To build and run dynamic advertisement application using default example and command:

```
$ cd <SDK_ROOT>/atmosic_sdk/platform/atm2/<platform>/examples/BLE_adv
$ make REF_BCN:=simple_beacon CFG_DYN_ADV:=1 run_all
```

It will enable the definition ***CFG_DYN_ADV*** and permit ***Advertising Parameter*** and ***Advertising DATA/ ScanRsp Payload*** dynamically change.

```
ifdef CFG_DYN_ADV
CFLAGS +=
        -DCFG_DYN_ADV
        -DCFG_ADV_CREATE_PARAM_CONST=0
        -DCFG_ADV_DATA_PARAM_CONST=0
endif
```

3.3.2 Update Advertisement Flow

After enabling the ***CFG_DYN_ADV*** definition, when the state machine transitions to ***S_ADV_STARTED*** state (refer to 3.2 Application State Machine), the ble_adv_start_on() function will do two things. One is calling update_adv_content() to update the payload, another is to start a 10 seconds timer.

The software timer id tid_update_adv is used to be a repeat start 10 seconds timer. That means that if the timer starts and counts to 10 seconds, it will start again and never stop.

Everytime the timer counts to 10 seconds, the update_adv_content() is executed.
The update_adv_content() function changes the Advertising Data type ***Complete Local Name*** in the ***Advertising Data/ ScanRsp Payload by*** calling atm_adv_set_adv_data() and atm_adv_set_scan_data(). The Software Development Kit (SDK) permits ***Advertising Data/ ScanRsp Payload*** to be updated in the ***S_ADV_STARTED*** state (refer to Figure 2 - Advertising State Transition Diagram).

The default ***Complete Local Name*** format is “Atmosic adv.count=xxx”, the xxx is a 3-digit decimal number. At the beginning, the number is 0, and it will be added 1 every time after executing update_adv_content() . Therefore, the ***Complete Local Name*** changes every 10 seconds, Table 2 shows the relationship between update counter number and the ***Complete Local Name***.

CONFIDENTIAL

Update Counter Number	Complete Local Name
0	Atmosic adv.count=000
1	Atmosic adv.count=001
255	Atmosic adv.count=255
256 (0)	Atmosic adv.count=000
1	Atmosic adv.count=001

Table 2 - Complete Local Name Update Rule

CONFIDENTIAL



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2020-2021 by Atmosic Technologies. All rights reserved.

www.atmosic.com